

An Embodied Squadron Commander for Long-Horizon LLM Teams Related Work, Protocol Design, and Auditable Evaluation Plan

AlemDICE Research Engineering Report

22 July 2026

Abstract

Language-model agents can interpret new objectives and improvise in unfamiliar worlds, but unconstrained peer discussion does not reliably produce one coherent team plan. This report develops a deliberately narrow alternative: one embodied agent is the squad commander and owns strategic task planning, while two wingmen retain tactical autonomy for movement, prerequisites, immediate survival, and execution. The commander is not a privileged fourth process and receives no global simulator state. A separate serial model call uses only the commander’s legal text observation and validated prior reports to issue a versioned, short-leased plan; the same three embodied agents then choose actions in parallel. The design is inspired by joint-intention and SharedPlans theories, practical teamwork models, role-based multi-agent learning, LLM supervisor–worker systems, embodied language planning, and distributed leases. It differs from all of these in an important way: the scientific treatment is an online authority and communication protocol layered onto an otherwise unchanged Alem Source policy. We specify failure semantics, information boundaries, metrics, preregistered 30-step and three-seed 200-step Luna studies, and a path to multiple squads. No new hierarchy efficacy result is claimed in this report; hosted-model outcomes remain pending until the staged gates and semantic trace audit complete.

1 Research question and thesis

The motivating question is not merely whether three LLM agents can exchange useful text. It is whether a team can maintain a coherent division of labor over many embodied decisions without silently changing the environment, action policy, or model underneath the comparison. Alem provides persistent, partially observable multi-agent interaction, differentiated physical specializations, networked language-model policies, exact trajectories, and achievement-level outcomes [2]. It therefore exposes a recurring gap between a plan that sounds coordinated and behavior that actually completes joint work.

The primary thesis is:

For one small squad, separating strategic authority from tactical execution can improve plan coherence and role persistence, provided that authority is explicit, plans expire, reports are validated, local action choice remains grounded in legal observations, and the serial planning overhead is measured rather than ignored.

“Squadron” is an organizational metaphor, not a claim that military doctrine is being reproduced. The operative abstraction is one commander and two wingmen in a single team. The commander has a body, fixed Alem specialization, observations, actions, rewards, and risks like every other player. It makes two logically separate calls when planning is due: first a planning-only call, then its ordinary embodied action call in parallel with the other two agents. The planner cannot act; the action call cannot revise the plan. Future work may compose several squads, but no cross-squad mechanism is part of the present claim.

Three observations motivate this direction. First, previous local peer-protocol pilots produced positive cohesion signals on isolated trajectories, but consensus often failed to become synchronized execution and prompt-based role auctions were fragile. Second, an earlier bodyless-leader treatment added a fourth logical participant and exposed every worker view to it; that is a useful comparison, but it is not the proposed embodied command structure. Third, the public Source behavior already has valuable properties—three concurrent action calls, affordance-grounded action parsing, private scratchpads, and one-tick peer broadcast—which should remain intact in the control arm.

2 Related work

2.1 Joint commitments and explicit team plans

Joint-intention theory treats teamwork as more than coincident individual goals: participants maintain commitments to a joint activity and have obligations when its status changes [3]. SharedPlans develops a richer account of partial plans, individual subplans, mutual belief, and the relationships among contributors to a complex group action [5]. These theories suggest why an LLM’s free-form “I will help” is too weak for measurement. It does not identify a plan version, task owner, dependency, success condition, or the point at which the statement ceases to authorize action.

STEAM translated teamwork theory into a reusable agent architecture for dynamic, uncertain domains. Its central contribution was not a particular domain plan but domain-independent rules for establishing and maintaining team activity under partial and inconsistent views [20]. Later decision-theoretic analysis formalized the trade-off between communication cost and coordination benefit [14]. The proposed squad protocol follows this tradition by making commitments and termination observable. It is intentionally less ambitious epistemically: it does not claim mutual belief or a formal team-optimal policy. A parsed record is evidence that a message satisfied a protocol, not that the model truly understood or obeyed it.

Contract Net is another relevant ancestor. It distributes tasks through announcement, bidding, award, and contractor execution rather than assuming a single fixed allocator [19]. That mechanism remains important for future dynamic team formation, but it did not work reliably in the initial prompt-only Alem role pilot: self-bids were poorly calibrated, awards could violate the visible ranking, and acknowledgements appeared without valid awards. The present method therefore tests fixed, explicit command authority before returning to negotiated formation. It narrows the causal question from “can every mechanism coexist?” to “does one valid planner plus bounded reports improve an otherwise matched team?”

2.2 Organization, hierarchy, and command failure

Multi-agent organizational research distinguishes hierarchies, teams, markets, coalitions, congregations, and other structures by authority, information flow, adaptability, and failure modes [7]. A hierarchy can reduce conflicting decisions and communication fan-out, but creates a bottleneck and a concentrated failure point. A team model can support richer mutual adjustment, but may spend more computation establishing common ground. This is why broadcast and star routing are separated from the authority treatment: the same commander plan should be testable with or without peer-visible statuses.

The design borrows the *expiration* idea from distributed leases. Gray and Cheriton used time-bounded rights so failures affect availability rather than allowing stale authority to persist indefinitely [4]. A squad task lease is not a distributed cache-consistency proof, and simulator ticks avoid the clock-skew assumptions of physical-time leases. The analogy nevertheless provides a clean safety rule: an invalid or missing review cannot silently renew old authority. If repeated invalid plans exhaust a lease, wingmen

stop pursuing the expired task and report that they are waiting. Version and team identifiers make delayed or cross-team records rejectable.

The first implementation deliberately has no failover. Promoting a wingman after commander failure would alter authority, introduce an election mechanism, and make the initial result harder to interpret. Commander inactivity, planning transport failure, invalid output, and plan expiry are logged separately. A provider transport failure retains the existing evaluator’s fail-and-resume semantics; a semantic planning failure preserves only the unexpired last valid plan.

2.3 Centralized training and hierarchical multi-agent learning

Centralized training with decentralized execution (CTDE) addresses learning non-stationarity by exposing a critic or value mixer to joint information during training while keeping deployed actors local. MADDPG learns centralized critics with per-agent actors [12]; QMIX factorizes a joint value subject to a monotonicity constraint that permits decentralized greedy action selection [17]. These are not direct instances of the proposed method. The squad commander performs *online* inference at execution time, changes the prompts presented to actors, and incurs serial latency and tokens. Calling it CTDE would hide precisely the intervention being measured.

Role-oriented MARL asks how roles can reduce a joint problem. ROMA learns identifiable emergent role representations [21]; RODE constructs a bi-level hierarchy in which a lower-frequency selector chooses restricted role action spaces and role policies operate at the primitive level [22]. This temporal decomposition strongly motivates a five-tick planning cadence, but there are two crucial differences. Alem roles such as warrior, forager, and miner are fixed environment capabilities, whereas squad assignments are temporary responsibilities. Moreover, the LLM planner emits language tasks rather than a learned latent role or restricted discrete action set. Compliance must therefore be measured from messages and behavior rather than guaranteed by policy architecture.

2.4 Supervisor–worker LLM systems and organizational topology

LLM multi-agent work shows several ways to impose structure. CAMEL uses role-playing and inception prompts to sustain cooperative conversations [10]. AutoGen exposes programmable conversational agents and interaction patterns [23]. MetaGPT encodes standardized workflows and assigns specialist roles in a software-production pipeline [6]. DyLAN dynamically selects agents and communication links for each task [11]. Collectively, these systems show that topology and role prompts are part of the algorithm, not incidental user-interface choices.

Most such evaluations are disembodied question answering, coding, or artifact production. A supervisor can often inspect a shared transcript or a worker’s finished artifact; an embodied commander instead has a partial local view, workers act simultaneously, and a plausible instruction may be physically impossible at that tick. The Alem method also requires a strict control: the baseline must not receive supervisor vocabulary, structured status demands, extra memory, or different sampling parameters. These constraints turn an orchestration pattern into a falsifiable behavioral treatment.

2.5 Embodied language planning and multi-robot coordination

Language-model planning for embodied agents reveals why strategy and affordance grounding should remain separate. Zero-shot planners can extract useful action knowledge but require admissibility filtering and replanning in the environment [8]. SayCan combines language-based task relevance with value-derived skill feasibility, preventing semantically appealing yet infeasible steps from dominating execution [1]. ReAct interleaves reasoning and action so environmental feedback can revise a plan [24]. In Alem, the Source affordance list and action validator continue to ground each physical action; the squad planner never bypasses them.

Area	Central mechanism	What transfers	What must not be conflated
Joint plans	commitments and subplans	explicit responsibility and termination	parsed text with mutual belief
CTDE	global training signal, local actors	compare centralized information assumptions	online planner with training-only critic
Hierarchical MARL	slow role selector, fast policies	temporal abstraction	latent role with language assignment
LLM workflows	supervisor and specialist agents	authority and topology as algorithm	shared artifact pipeline with embodied control
Memory systems	select, summarize, retrieve	bounded context state	memory tier with command authority
Distributed leases	expiring rights	stale-authority containment	simulator protocol with cache-consistency proof

Table 1: Conceptual distinctions needed for a precise research claim.

SMART-LLM decomposes instructions, forms coalitions, and allocates work for multiple robots [9]. RoCo lets robot-specific LLM agents discuss strategies and combines their high-level output with motion planning and environment feedback [13]. Both support the premise that language models can help with multi-robot task structure. Our experiment asks a different question: under matched repeated interaction and a fixed team, is a single local commander more reliable and economical than leaving strategic planning to all three agents? The answer cannot be inferred from one-shot plan quality.

2.6 Memory, context, and long-horizon role coherence

Long missions create two distinct memory problems. An agent must remember world facts that have left its observation window, and it must retain the currently authorized responsibility without letting old instructions compete with new ones. Generative Agents uses an experience stream, retrieval, reflection, and planning to maintain behavior over time [16]. Reflexion stores linguistic feedback in episodic memory to improve later decisions [18]. MemGPT treats the context window as a managed memory hierarchy with explicit movement between tiers [15].

The first squad treatment does not add a retrieval database. It keeps Source’s private worker scratchpad unchanged, adds a bounded planner-only scratchpad, and places a compact authoritative plan in every current action prompt. Plan version, lease, and assignment are middleware state rather than memories the model must reconstruct. This distinction matters: a *memory hierarchy* manages which information is available, while a *command hierarchy* determines who may change team objectives. Future experiments can cross these factors instead of assuming that better memory automatically produces role compliance.

2.7 Resulting gap

The literature establishes strong components but not the full target combination: one embodied, locally informed commander; unchanged concurrent action policies; atomic structured plans; explicit authority; bounded leases; validated delayed reports; and paired long-horizon evaluation against a source-matched control. Table 1 states the main boundaries.

3 Method: one embodied squad

3.1 Participants and information boundary

Let the physical team be $M = \{0, 1, 2\}$ and let $c \in M$ be configurable. The initial study fixes $c = 0$. There are exactly three action clients and three trajectory slots. When planning is due, a separate client instance is cloned from client c with the same provider, model, generation settings, timeout, retries, and context policy, but a planner-specific prompt-cache suffix. Its usage is attributed to agent c under a distinct planning phase.

At tick t , the planner input is limited to:

1. the public game rules;
2. agent c 's legal long- and short-term text observation at t ;
3. the last accepted plan and its metadata;
4. a bounded private planner scratchpad; and
5. previously validated SCP1 reports from members.

It does not receive other agents' direct observations, raw JAX state, reward internals, analysis-only opportunity labels, worker scratchpads, or worker reasoning. This is a local commander, not an omniscient coordinator.

3.2 Plan schema and state machine

A planning response contains one tagged JSON object. **REPLACE** requires a nonempty objective and exactly one assignment for every member. An assignment contains **agent_id**, unique **task_id**, **directive**, **target**, observable **completion**, dependency task IDs, and an optional synchronization action/window. Dependencies must resolve within the plan and form an acyclic graph. Synchronization actions must be canonical Alem actions and their windows must fit inside the lease. Validation is atomic: one malformed assignment rejects the proposal.

Middleware, not the model, adds team ID, monotonic version, issue tick, next review tick, and expiry tick. A scheduled review may return **KEEP** only when a plan is active; this preserves the version and renews the lease. Event reviews and invalid-plan retries require **REPLACE**. An invalid response does not change or extend the prior plan. If no unexpired plan remains, agents may take an immediately necessary survival action; otherwise they choose **Noop** and report **WAITING**.

Planning occurs at tick zero, every five ticks, on the next tick after a new **BLOCKED**, **COMPLETE**, or **EMERGENCY** transition, and on the next tick after invalid plan output. Repeated identical status states are deduplicated. Unscheduled replans are capped at $\lceil T/10 \rceil$, yielding at most nine plan calls in a 30-step episode and 60 in a 200-step episode. A scheduled call may absorb a pending event without adding an extra call, but it remains an event review and cannot use **KEEP**.

3.3 Execution authority

After validation, the full accepted plan is placed in all three action prompts before the same tick's parallel action calls; each prompt highlights its owner's assignment. This shared plan view prevents dependencies from becoming opaque. Only the planning phase can change objectives or assignments. The commander's action phase executes its own task and cannot revise the plan. Wingmen may choose local routes, acquire prerequisites, react to immediate danger, and determine feasible primitive actions, but may not create competing objectives, reassign members, or treat scratchpad plans as team authority.

This is a prompt-level behavioral contract, not forced action substitution. The existing action generator, visible reasoning behavior, scratchpad, affordance display, semantic-retry policy, parser, and environment validity check remain in the path. Consequently, failure to follow an assignment remains observable and scientifically meaningful rather than being hidden by middleware-selected actions.

3.4 Status records and routing

Agents use the existing optional `<communication>` field for one bounded record:

```
SCP1|TEAM=squad-0|VERSION=3|TASK=ore-2|STATE=ACTIVE|EVIDENCE=at iron vein|REQUEST=wood
```

Allowed states are `ACK`, `ACTIVE`, `BLOCKED`, `COMPLETE`, `EMERGENCY`, and `WAITING`. The ledger rejects malformed records, wrong team IDs, stale versions, and tasks not assigned to the sender. Rejected records remain in the raw audit trail but cannot trigger planning. A worker’s attempt to embed a plan is counted and never accepted.

`COMMANDER-BROADCAST` retains Source’s one-tick peer broadcast and also enters valid statuses in the commander ledger. `COMMANDER-STAR` gives the same planner identical reports and plan timing, but does not deliver wingman statuses to peers. This isolates routing from strategic authority. `SOURCE` retains ordinary free broadcast and has no commander instructions, schema, or planner call.

3.5 Failure semantics

Failure	Runtime response	Scientific record
Provider transport error	fail episode; preserve attempt; resume seed later	attempts, error types, requests, billed usage
Invalid plan JSON/schema	retain only unexpired plan; retry next tick within cap	raw output, rejection, unchanged expiry
Invalid <code>KEEP</code>	reject; require replacement	invalid-keep counter
Expired plan	remove authority; survival/Noop and <code>WAITING</code>	uncovered tick and expiry counter
Malformed/stale status	audit but do not enter active ledger or trigger review	reason-specific rejection counter
Commander inactive	planner still uses its legal inactive text; action constraints remain	inactivity and plan behavior visible
Commander unavailable	no failover in v1	explicit single-point failure
Wingman noncompliance	do not override primitive action	assignment/action trace for semantic review

4 Implementation plan and delivered architecture

The implementation is additive. Existing `baseline`, `leader_peer`, and `leader_no_peer` behavior is preserved; the new topologies are `embodied_commander_broadcast` and `embodied_commander_star`. The one-team configuration is already namespaced by `team.id` so later squads do not require rewriting the record format.

The following invariants are release blockers:

1. **Control preservation.** A scripted `SOURCE` episode and a scripted commander episode with identical primitive actions have identical observations, actions, rewards, dones, and physical trajectory dimensions.
2. **No fourth participant.** Commander treatments report three physical and three logical participants; planning usage belongs to the selected embodied commander.
3. **Same-tick order.** An accepted plan is visible before all three action calls, and a concurrency barrier confirms those calls overlap.

Component	Responsibility
<code>team_commander.py</code>	strict plan/status parsing, dependency validation, leases, triggers, authority ledger, planner prompt, directives, metrics
<code>AgentFactory</code>	clone commander client without adding a fourth client; isolate cache namespace
<code>evaluator.py</code>	serial due-plan phase, same-tick directive delivery, three parallel action calls, routing and durable accounting
<code>robust.all.py</code>	treatment-only authority/status instructions; Source prompt unchanged
Hydra profiles	exact Luna Source settings with dedicated 30/200 seeds and caps
Study launcher	preflight, dry run, immutable manifest, hashes, call ceilings, resume, staged arms
Study summarizer	paired episode table, attempt-ledger usage, explicit availability, preregistered gates
Tests	parser/state properties, routing identities, leak sentinels, parallel ordering, trajectory parity, profile and cache checks

Table 3: Implementation boundaries. File names are relative to the AlemDICE LLM baseline.

4. **No hidden-state path.** Sentinel raw-state fields and non-commander legal views never appear in the planner prompt.
5. **Atomic authority.** Invalid plans, stale statuses, wrong tasks, and worker-authored plans cannot modify the active ledger or extend a lease.
6. **Auditability.** Resolved base/per-arm configuration hashes, prompt contract hash, source and lock hashes, cache keys, exact commands, attempts, failures, and usage are durable.

5 Evaluation design

5.1 Arms and held-constant variables

The initial paid experiment contains two arms only:

Source: current Source protocol: three `robust.all` agents, ordinary peer broadcast, private scratchpads, and concurrent actions.

Commander–Broadcast: the full structured treatment: Agent 0 planner, leased plan, executor authority, validated SCP1 statuses, and retained peer broadcast.

The model is `gpt-5.6-luna` through the OpenAI Responses API, with temperature 1.0, reasoning effort `none`, 8192 output tokens, disabled storage, three prompt-cache traffic shards, explicit 30-minute cache mode, 420-second timeout, five transport attempts, and identical worker role prompts. The environment, difficulty, seed, action code, context lengths, rendering, retry policy, and debrief setting are matched. Cache namespaces differ by arm to prevent a condition from accidentally inheriting another condition’s cached prefix accounting.

COMMANDER–STAR is deferred until broadcast passes. Commander IDs 1 and 2 are then tested on the winning routing to determine whether results depend on Agent 0’s warrior specialization or map position. This ordering avoids spending the initial budget on a three-way interaction before the core authority mechanism is known to work.

5.2 Stage 0: deterministic and provider preflight

Static and scripted checks precede paid calls. The full test suite and Ruff must pass; the two new profiles must compose; launcher dry runs must create no files; and the deterministic smoke episode must preserve the Source trajectory contract. The hosted preflight makes exactly two bounded Luna requests: one response that must contain a parseable `Noop` action and one response that must contain a valid three-member `REPLACE` plan. A preflight is compatibility evidence, not behavioral evidence.

5.3 Stage 1: 30-step wiring and qualitative gate

Both arms run Easy seed 12000 for 30 ticks. Promotion requires:

- both canonical episodes complete with zero transport errors;
- at least one valid treatment plan;
- active plan coverage ≥ 0.80 ;
- action parse rate ≥ 0.95 and attempted-status parse rate ≥ 0.90 ;
- zero accepted unauthorized plans, zero accepted stale assignments, and zero construction-level hidden-state guard violations; and
- manual review of planner and action traces, including one invalid or transition case if present.

Passing this stage means the mechanism is executable and auditable. A return or achievement difference at one short seed is not an efficacy result.

5.4 Stage 2: three paired seeds at 200 steps

Before inspection, both arms are frozen on Easy seeds 12000–12002 and 200 ticks. The exploratory promotion rule requires all three pairs valid, achievement rate higher on at least two seeds, an action-parse decline larger than two percentage points on at most one seed, at least 90% plan coverage, at least 90% valid plan calls and attempted status reports, zero accepted authority/staleness violations, and no more than 35% mean regression in either total tokens or wall time per step. All seed-level values are reported even if the aggregate gate fails.

Three seeds are too few for a confirmatory claim. This stage decides whether the mechanism merits the next ablations, diagnoses overhead, and estimates variance. If it fails, revisions move to new development seeds and receive a new protocol version; the original outputs remain immutable.

5.5 Stage 3 and confirmatory expansion

If broadcast passes, run star routing on the same development seeds, then rotate the commander identity. Freeze the winning combination and evaluate at least 20 held-out paired seeds per Easy, Medium, and Hard condition. Power calculations should use the observed paired variance, not an arbitrary universal seed count. Report paired bootstrap intervals for continuous outcomes, exact paired results for sparse binary achievements where appropriate, and a hierarchical model with seed and difficulty effects. No across-difficulty average replaces the three difficulty-specific estimates.

Long-horizon evaluation crosses 200, 500, 1000, and environment-terminal caps. It should measure role consistency inside opportunity windows, stale-task execution after replacement, dependency satisfaction, synchronized-action success, blocker recovery, plan churn, and assignment switches. A later factorial experiment crosses command structure with typed event memory, because authority and memory are separate hypotheses.

5.6 Metrics and causal diagnostics

Task outcomes include team return, total/normal/coordination achievement rates, deaths and survival, valid transfers and requests, successful synchronized actions, and episode length. Protocol metrics include:

- plan-call validity, KEEP/REPLACE counts, coverage, age, expiry, version churn, and assignment switches;
- assignment acknowledgement rate and latency;
- status validity, state distribution, stale/wrong-task rejection, waiting, and blocker/completion/emergency-to-replan latency;
- unauthorized plan attempts and the stronger invariant that zero are accepted; and

- semantic compliance sampled from action traces: whether behavior advances the assigned task when locally feasible.

Efficiency metrics separate action calls from commander-plan calls, logical responses from provider attempts, input/output/reasoning/cache tokens, emitted payload bytes from fan-out delivery bytes, serial plan latency from parallel worker-round latency, and total wall time per tick. The sum of model-call latencies is not substituted for wall time because calls overlap. Failed attempts remain in cost/error accounting while invalid episodes are excluded from reward aggregates.

The key causal diagnostics are as important as the headline score. If plan validity is high but coverage is low, lease/review logic is at fault. If coverage is high but acknowledgement is low, instructions may not be salient. If acknowledgement is high but assigned behavior is absent, the result is linguistic compliance without execution. If achievement improves but token/wall cost breaks the gate, the mechanism is effective but not scalable. If star changes behavior while plan traces remain fixed, peer status visibility—not command authority—is the likely cause.

6 Extension to multiple squads

The current schema includes `team_id` now, but implementing several squads is intentionally deferred. A scalable extension requires decisions that the one-squad study cannot answer:

1. whether squads are fixed or formed from capability evidence;
2. whether commanders negotiate peer-to-peer, elect a mission commander, or use a task market;
3. which status summaries cross squad boundaries and at what rate;
4. how cross-squad dependencies, leases, and cancellation propagate; and
5. what happens when a commander fails, partitions, or equivocates.

A conservative design is a two-level namespace. Each local commander owns only its squad plan; an inter-squad record may request or promise a capability but cannot directly assign another squad’s wingmen. Cross-squad work uses explicit dependencies and leases, and raw local observations never flow automatically to another planner. Evaluate $N = 3, 6, 12, 24$ agents with one squad per three agents, then compare commander mesh, sparse task-scoped routing, and a bodyless mission planner as separate treatments. Communication cost must be normalized by deliveries rather than emitted messages.

7 Threats to validity and falsification criteria

The most serious threat is treatment bundling. The full intervention adds a planner call, structured plan, authority instruction, status format, and event logic. The first two-arm study tests the package. Subsequent ablations are needed to locate causality, but adding a prompt-only intermediate arm to the first paid stage would dilute the locked question and budget.

Model stochasticity and three development seeds limit inference. Agent 0 has a particular specialization and spawn state. Structured output may consume tokens or distract from action tags. A strict parser can undercount semantically useful free text, while a permissive parser can inflate compliance. Status evidence is self-reported; validation establishes authority and freshness, not truth. A commander may hallucinate a coherent but infeasible plan from its partial view. Some episodes may contain few opportunities for genuinely joint achievements. Manual semantic review is therefore required alongside automated metrics.

The hypothesis is falsified, rather than rescued by post-hoc narrative, if the frozen treatment cannot maintain plan/status validity, causes material action format degradation, produces assignments that are acknowledged but not executed, fails to improve held-out outcomes, or exceeds the preregistered overhead bound. Positive plan-coherence metrics with no task effect would support the mechanism claim

but not the performance claim. Positive results only on commander ID 0 would indicate specialization or position dependence, not a general hierarchy effect.

8 Current status and reproducibility contract

The implementation branch contains the new state machine, runtime integration, profiles, launcher, summarizer, tests, and this report. New hierarchy outcomes are deliberately marked *pending*. Each paid study root is required to contain source and lock hashes, resolved base and per-arm configuration hashes, the prompt-contract hash, exact commands and cache keys, seed/call ceilings, canonical episode artifacts, append-only attempt ledgers, debug traces, a machine summary, and a Markdown report. A dirty launch requires an explicit override and records the status; existing outputs are never relabeled as a new protocol.

The execution order is:

```
./commands.sh commander-study --stage 30 --dry-run
./commands.sh commander-study --stage 30 --preflight
./commands.sh commander-study --stage 30
# inspect automated gate and semantic traces
./commands.sh commander-study --stage 200 --dry-run
./commands.sh commander-study --stage 200
```

The final two commands are run only after the short gate. `--include-star` adds the routing ablation on a fresh study; it does not alter the preregistered Source-versus-broadcast contrast.

9 Conclusion

The best first hierarchy experiment is not a larger committee of planners. It is a small, auditable separation of concerns: one locally informed embodied commander owns a short-lived whole-team plan; every agent, including the commander, executes through the unchanged grounded action policy; wingmen retain tactical autonomy; and validated reports trigger bounded replanning. Classical teamwork theory supplies the commitment vocabulary, MARL supplies temporal role abstraction, LLM systems supply flexible semantic decomposition, embodied planning supplies the grounding warning, and distributed leases supply a useful stale-authority discipline. The staged experiment is designed to discover whether these pieces produce actual collective performance rather than merely better-formatted agreement.

References

- [1] Michael Ahn, Anthony Brohan, Noah Brown, et al. Do as i can, not as i say: Grounding language in robotic affordances. In *Proceedings of the 6th Conference on Robot Learning*, volume 205 of *Proceedings of Machine Learning Research*, pages 287–318, 2023. URL https://openreview.net/forum?id=bdHkMjBJG_w.
- [2] Alem Team. Alem: A simulation environment for networked ai models. *arXiv preprint arXiv:2606.08340*, 2026. URL <https://arxiv.org/abs/2606.08340>.
- [3] Philip R. Cohen and Hector J. Levesque. Teamwork. *Noûs*, 25(4):487–512, 1991. doi: 10.2307/2216075.
- [4] Cary G. Gray and David R. Cheriton. Leases: An efficient fault-tolerant mechanism for distributed file cache consistency. In *Proceedings of the Twelfth ACM Symposium on Operating Systems*

- Principles*, pages 202–210, 1989. doi: 10.1145/74850.74870. URL <https://web.stanford.edu/class/cs240/readings/leases.pdf>.
- [5] Barbara J. Grosz and Sarit Kraus. Collaborative plans for complex group action. *Artificial Intelligence*, 86(2):269–357, 1996. doi: 10.1016/0004-3702(95)00103-4. URL <https://dash.harvard.edu/handle/1/2562070>.
 - [6] Sirui Hong, Mingchen Zhuge, Jonathan Chen, et al. MetaGPT: Meta programming for a multi-agent collaborative framework. In *International Conference on Learning Representations*, 2024. URL https://proceedings.iclr.cc/paper_files/paper/2024/hash/6507b115562bb0a305f1958ccc87355a-Abstract-Conference.html.
 - [7] Bryan Horling and Victor Lesser. A survey of multi-agent organizational paradigms. *The Knowledge Engineering Review*, 19(4):281–316, 2004. doi: 10.1017/S0269888905000317.
 - [8] Wenlong Huang, Pieter Abbeel, Deepak Pathak, and Igor Mordatch. Language models as zero-shot planners: Extracting actionable knowledge for embodied agents. In *Proceedings of the 39th International Conference on Machine Learning*, volume 162 of *Proceedings of Machine Learning Research*, pages 9118–9147, 2022. URL <https://proceedings.mlr.press/v162/huang22a.html>.
 - [9] Shyam Sundar Kannan, Vishnunandan L. N. Venkatesh, and Byung-Cheol Min. SMART-LLM: Smart multi-agent robot task planning using large language models. *arXiv preprint arXiv:2309.10062*, 2023. URL <https://arxiv.org/abs/2309.10062>.
 - [10] Guohao Li, Hasan Abed Al Kader Hammoud, Hani Itani, Dmitrii Khizbullin, and Bernard Ghanem. CAMEL: Communicative agents for “mind” exploration of large language model society. In *Advances in Neural Information Processing Systems*, volume 36, 2023. URL <https://proceedings.neurips.cc/paper/2023/hash/a3621ee907def47c1b952ade25c67698-Abstract-Conference.html>.
 - [11] Zijun Liu, Yanzhe Zhang, Peng Li, Yang Liu, and Diyi Yang. A dynamic llm-powered agent network for task-oriented agent collaboration. *arXiv preprint arXiv:2310.02170*, 2023. URL <https://arxiv.org/abs/2310.02170>.
 - [12] Ryan Lowe, Yi Wu, Aviv Tamar, Jean Harb, Pieter Abbeel, and Igor Mordatch. Multi-agent actor-critic for mixed cooperative-competitive environments. In *Advances in Neural Information Processing Systems*, volume 30, 2017. URL https://proceedings.neurips.cc/paper_files/paper/2017/hash/68a9750337a418a86fe06c1991a1d64c-Abstract.html.
 - [13] Zhao Mandi, Shreeya Jain, and Shuran Song. RoCo: Dialectic multi-robot collaboration with large language models. *arXiv preprint arXiv:2307.04738*, 2023. URL <https://arxiv.org/abs/2307.04738>.
 - [14] Ranjit Nair and Milind Tambe. The communicative multiagent team decision problem: Analyzing teamwork theories and models. *Journal of Artificial Intelligence Research*, 23:367–420, 2005. doi: 10.1613/jair.1547. URL <https://www.jair.org/index.php/jair/article/view/10471>.
 - [15] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. MemGPT: Towards llms as operating systems. *arXiv preprint arXiv:2310.08560*, 2023. URL <https://arxiv.org/abs/2310.08560>.
 - [16] Joon Sung Park, Joseph O’Brien, Carrie Jun Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*, pages 1–22, 2023. doi: 10.1145/3586183.3606763.

- [17] Tabish Rashid, Mikayel Samvelyan, Christian Schroeder, Gregory Farquhar, Jakob Foerster, and Shimon Whiteson. QMIX: Monotonic value function factorisation for deep multi-agent reinforcement learning. In *Proceedings of the 35th International Conference on Machine Learning*, volume 80 of *Proceedings of Machine Learning Research*, pages 4295–4304, 2018. URL <https://proceedings.mlr.press/v80/rashid18a.html>.
- [18] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 36, 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/1b44b878bb782e6954cd888628510e90-Abstract-Conference.html.
- [19] Reid G. Smith. The contract net protocol: High-level communication and control in a distributed problem solver. *IEEE Transactions on Computers*, C-29(12):1104–1113, 1980. doi: 10.1109/TC.1980.1675516.
- [20] Milind Tambe. Towards flexible teamwork. *Journal of Artificial Intelligence Research*, 7:83–124, 1997. doi: 10.1613/jair.433.
- [21] Tonghan Wang, Heng Dong, Victor Lesser, and Chongjie Zhang. ROMA: Multi-agent reinforcement learning with emergent roles. In *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pages 9876–9886, 2020. URL <https://proceedings.mlr.press/v119/wang20f.html>.
- [22] Tonghan Wang, Tarun Gupta, Anuj Mahajan, Bei Peng, Shimon Whiteson, and Chongjie Zhang. RODE: Learning roles to decompose multi-agent tasks. In *International Conference on Learning Representations*, 2021. URL <https://openreview.net/forum?id=TTUVg6vkNjK>.
- [23] Qingyun Wu, Gagan Bansal, Jieyu Zhang, et al. AutoGen: Enabling next-gen llm applications via multi-agent conversation. *arXiv preprint arXiv:2308.08155*, 2023. URL <https://arxiv.org/abs/2308.08155>.
- [24] Shunyu Yao, Jeffrey Zhao, Dian Yu, et al. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023. URL https://openreview.net/forum?id=WE_vluYUL-X.